The History of DevOps

Shinsato, Miles

Bellevue University, CSD-380

Assignment 1.3

October 24, 2026

The History of DevOps

Introduction

Around the mid to late 2000s, a trend began to emerge in the software development community, frustration to inefficient or ineffective practices. DevOps was theorized as a way to help address challenges found in software development and operations. DevOps is not so much a single practice, but a collection of practices. Three of the most influential movements were the Lean Movement, the Agile Manifesto, and the Continuous Delivery Movement. Each of these contributed to what is now recognized as DevOps through an emphasis on efficient, collaborative, and constant improvements throughout a process.

I consider the most insightful piece about DevOps is that it is truly about a change in thought processes. It isn't about getting the newest technology, but changing how a team works, communicates, and influence each other in the workplace. At its core, a team of people that can function at a high level together will outperform any team of the best individual contributors that do not mesh well with each other. Additionally, while technology will continuously evolve, the people and processes behind it must also be willing to evolve as well.

DevOpsDays is what truly helped popularize the movement. In 2009, the first DevOpsDays conference was held in Ghent, Belgium. This was founded by Belgian IT consult, Patrick Debois, who has since co-authored the book on DevOps, the DevOps Handbook. Since 2009, DevOpsDays has become an event that happens in multiple countries, multiple times a year.

The Lean Movement

The movement I want to highlight first is the Lean Movement. While the Lean Movement has its origins in the automotive industry with Ford and Toyota (Lean Enterprise

Institute, n.d.) in the early to mid 1900s, the practices translate well to software development. The core practices of the Lean methodology break down into five areas: identify value, map the Value Stream, create flow, establish pull, and seek perfection. (Lean Enterprise Institute, n.d.). These are guiding principles rather than a strict sequence of steps, and they help organizations identify the needs of customers, visualizing the process of delivering value, eliminating waste, and improving quality and efficiency (Atlassian, n.d.). It's easy to see how this can be translated from the automotive industry and into something like software development.

There are two core pieces from the Lean methodology: continuous improvement and respect for the people doing the work (Atlassian, n.d.). It challenges people to not accept an inefficient process, but to find ways to optimize the processes after identifying the issue. Lean prioritizes regularly finding friction points and seeking ways to solve them at its root while understanding the way it affects those who are doing the work to make things progress.

These principles helped influence DevOps by encouraging teams to improve the entire software delivery process instead of focusing on individual tasks. I find this especially important since working harder does not mean working efficiently. Often time, improving a process, while sometimes time consuming, can produce a better workflow in the long run, being much more efficient overall and delivering a better result.

The Agile Manifesto

Looking at another major influence on DevOps would be the Agile Manifesto. Its origins come from a group of seventeen software development professionals that met in 2001 at a ski resort in Utah to discuss better approaches to developing software. (*Manifesto for Agile Software Development*, 2001). The Agile Manifesto had four core focuses to it: individuals and

interactions, working software, customer collaboration, and responding to change. These seem simple enough, but the importance of those guiding lights cannot be understated.

Agile encourages developers to work in smaller increments, seek feedback, and adapt as the feedback comes in. The earlier feedback happens the better, since it makes it immensely easier to change and home in on what is needed. Imagine a person was tasked to paint a room blue and is devasted to find out later that they painted the room baby blue instead of the intended navy blue because of vague direction from the customer. This would need the job to be completely redone, but if they were told the correct color after finishing one wall, it would have saved a lot of time and effort. This is why early, and constant feedback is crucial in software development.

The communication and collaboration focus are what I personally find the most important from the Agile Manifesto. Having the collaborative mindset, which DevOps extended towards both software development and IT operations is better than any tool or technology. This allows teams to work together throughout the software lifecycle instead of treating development and deployment as separate responsibilities.

The Continuous Delivery Movement

With the focus on Agile practices, organizations also needed to be able to efficiently and reliably test the frequent releases and updates. Continuous Delivery helps address this challenge. AWS explained that “with continuous delivery, every code change is built, tested, and then pushed to a non-production testing or staging environment. There can be multiple, parallel test stages before a production deployment” (AWS, n.d. -a). Through Continuous Delivery, teams use automation to test software at multiple stages before it ever reaches the end user. This helps identify problems earlier and keeps updates ready for release when needed. Since there can be

multiple updates moving through the testing and staging process, teams can maintain a steady flow of software that is ready to go when needed.

A part that can be confusing is there is a difference between continuous delivery and continuous deployment. Continuous deployment automatically releases deployments on code changes to production when certain criteria, such as passing tests, are met. This would mean that changes reach end users, allowing for real-world usage, without the need for manual approval for each deployment. Continuous delivery uses automated processes to build and test updates, keeping the software ready to be released. Think of this as a prep and stage zone for updated code is checked and kept ready for future deployment.

Continuous delivery I feel is important to practice for several reasons, but there are two in particular that I find most beneficial. The first is that it uses automation to test, integrate code, and prepare it for deployment. This can reduce repetitive manual work and allow developers to focus on other tasks, which is my second reason I feel that continuous delivery is good practice. AWS has highlighted a few other reasons that continuous delivery is beneficial. They stated that continuous delivery can improve code quality and help deliver updates faster. (AWS, n.d.-b).

Conclusion

The history of DevOps shows how different movements helped shape the way software developed, tested, and delivered today. The Lean Movement introduced the importance of eliminating waste and continuously improving processes. The Agile Manifesto emphasized communication, collaboration, and adapting to change. While the Continuous Delivery Movement focused on using automation to make software releases more reliable and efficient. Together, these ideas helped establish the foundation for modern DevOps.

Of these principles, I feel that communication and collaboration to be the most important. New technologies and automation can make processes more efficient and effective. However, they cannot replace a team that works well together and are willing to not only learn from mistakes but also teach each other. Additionally, this is something that I feel applies to more than just software development. In any workplace, people who communicate effectively, share knowledge, and continuously look for ways to do better together will always be able to accomplish more. DevOps is a good example of how improving the way people work is just as important as the tools they use.

References

Atlassian. (n.d.). *What is Lean Methodolgy?* Retrieved October 25, 2026, from https://www.atlassian.com/agile/project-management/lean-methodology

AWS. (n.d.-a). *Benefits of Continuous Delivery*. Retrieved October 25, 2026, from https://docs.aws.amazon.com/whitepapers/latest/practicing-continuous-integration-continuous-delivery/benefits-of-continuous-delivery.html

AWS. (n.d.-b). *What is Continuous Delivery?* Retrieved October 25, 2026, from https://aws.amazon.com/devops/continuous-delivery/

Buchanan, I. (n.d.). *History of DevOps*. Atlassian. Retrieved October 25, 2026, from https://www.atlassian.com/devops/what-is-devops/history-of-devops

Dobra, A. (2025, March 31). History of DevOps – When Did DevOps Become a Thing? *Bunnyshell*. Retrieved October 25, 2026, from https://www.bunnyshell.com/blog/history-of-devops/

GeekForGeeks. (2026, February 27). *The evolution of DevOps 3 major trends for future*. GeeksforGeeks. Retrieved October 25, 2026, from https://www.geeksforgeeks.org/devops/the-evolution-of-devops-major-trends-for-future/

Lean Enterprise Institute. (2023, March 31). *A brief history of lean - lean Enterprise Institute*. https://www.lean.org/explore-lean/a-brief-history-of-lean/

Manifesto for Agile software development. (n.d.). Retrieved October 25, 2026, from https://agilemanifesto.org/